

Bubble Sort – Sommer 2020

Pseudocode (IHK-Stil):

```
Funktion bubbleSort(A: Array<Integer>)  
  for i = 0 to länge(A) - 2  
    for j = 0 to länge(A) - i - 2  
      if A[j] > A[j+1] dann  
        tausche(A[j], A[j+1])  
      end if  
    end for  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void bubbleSort(int[] A) {  
  for (int i = 0; i < A.length - 1; i++) {  
    for (int j = 0; j < A.length - i - 1; j++) {  
      if (A[j] > A[j + 1]) {  
        int tmp = A[j];  
        A[j] = A[j + 1];  
        A[j + 1] = tmp;  
      }  
    }  
  }  
}
```

Erklärung:

Ablauf: In jeder Runde wird das größte Element „nach rechts geschoben“.

Beispiel: [5,3,1] → Runde1: [3,1,5], Runde2: [1,3,5].

Komplexität: $O(n^2)$.

Typische Fehler: Schleifenindizes falsch gesetzt.

Prüfungstipp: Nach jedem Durchlauf ist das größte Element am Ende.

Insertion Sort – Winter 2020/21

Pseudocode (IHK-Stil):

```
Funktion insertionSort(A: Array<Integer>)  
  for i = 1 to lange(A)-1  
    key = A[i]  
    j = i - 1  
    while j ≥ 0 und A[j] > key  
      A[j+1] = A[j]  
      j = j - 1  
    end while  
    A[j+1] = key  
  end for  
end Funktion
```

Java-Beispiel:

```
public static void insertionSort(int[] A) {  
  for (int i = 1; i < A.length; i++) {  
    int key = A[i];  
    int j = i - 1;  
    while (j >= 0 && A[j] > key) {  
      A[j+1] = A[j];  
      j--;  
    }  
    A[j+1] = key;  
  }  
}
```

Erklärung:

Ablauf: Elemente werden eins nach dem anderen an die richtige Stelle eingefügt.

Beispiel: [5,2,4,1] → nach Schritt1: [2,5,4,1] → Schritt2: [2,4,5,1] → Schritt3: [1,2,4,5].

Komplexität: $O(n^2)$.

Gut für kleine oder fast sortierte Arrays.

Typische Fehler: Falscher Index bei j+1.

Prüfungstipp: Immer „einsortieren“ betonen.

Selection Sort – Sommer 2019

Pseudocode (IHK-Stil):

```
Funktion selectionSort(A: Array<Integer>)
  for i = 0 to länge(A)-2
    minIndex = i
    for j = i+1 to länge(A)-1
      if A[j] < A[minIndex] dann
        minIndex = j
      end if
    end for
    tausche(A[i], A[minIndex])
  end for
end Funktion
```

Java-Beispiel:

```
public static void selectionSort(int[] A) {
  for (int i = 0; i < A.length - 1; i++) {
    int minIndex = i;
    for (int j = i + 1; j < A.length; j++) {
      if (A[j] < A[minIndex]) {
        minIndex = j;
      }
    }
    int tmp = A[i];
    A[i] = A[minIndex];
    A[minIndex] = tmp;
  }
}
```

Erklärung:

Ablauf: Sucht das kleinste Element und tauscht es an die richtige Stelle.

Beispiel: [64,25,12,22,11] → kleinste=11 → [11,25,12,22,64] → weiter so bis sortiert.

Komplexität: $O(n^2)$.

Typische Fehler: Vergessen zu tauschen.

Prüfungstipp: Schlüsselwort „kleinstes Element suchen und tauschen“.

MergeSort (rekursiv) – Seite 1 (Code)

Pseudocode (IHK-Stil):

```
Funktion mergeSort(A: Array<Integer>)  
  if länge(A) > 1 dann  
    mitte = länge(A) / 2  
    L = kopiere(A[0..mitte-1])  
    R = kopiere(A[mitte..ende])  
    mergeSort(L)  
    mergeSort(R)  
    merge(A, L, R)  
  end if  
end Funktion
```

Java-Beispiel:

```
import java.util.Arrays;  
  
public static void mergeSort(int[] A) {  
  if (A.length > 1) {  
    int mid = A.length / 2;  
    int[] L = Arrays.copyOfRange(A, 0, mid);  
    int[] R = Arrays.copyOfRange(A, mid, A.length);  
    mergeSort(L);  
    mergeSort(R);  
    merge(A, L, R);  
  }  
}  
  
public static void merge(int[] A, int[] L, int[] R) {  
  int i=0, j=0, k=0;  
  while (i<L.length && j<R.length) {  
    if (L[i] <= R[j]) A[k++] = L[i++];  
    else A[k++] = R[j++];  
  }  
  while (i<L.length) A[k++] = L[i++];  
  while (j<R.length) A[k++] = R[j++];  
}
```

Erklärung:

→ Siehe nächste Seite für Ablauf-Erklärung mit Beispiel.

MergeSort (rekursiv) – Seite 2 (Erklärung)

Erklärung:

Ablauf: Teile das Array in zwei Hälften, sortiere jede Hälfte rekursiv, und führe sie wieder zusammen.

Beispiel: [38,27,43,3,9,82,10]

1. Teile: [38,27,43] und [3,9,82,10]

2. Sortiere links: [27,38,43], rechts: [3,9,10,82]

3. Merge: [3,9,10,27,38,43,82]

Komplexität: $O(n \log n)$.

Typische Fehler: Merge-Funktion vergessen oder falsch zusammengeführt.

Prüfungstipp: Immer „Teilen – Sortieren – Zusammenführen“ sagen.

QuickSort (rekursiv) – Seite 1 (Code)

Pseudocode (IHK-Stil):

```
Funktion quickSort(A: Array<Integer>, low: Integer, high: Integer)
  if low < high dann
    pi = partition(A, low, high)
    quickSort(A, low, pi-1)
    quickSort(A, pi+1, high)
  end if
end Funktion
```

Java-Beispiel:

```
public static void quickSort(int[] A, int low, int high) {
  if (low < high) {
    int pi = partition(A, low, high);
    quickSort(A, low, pi - 1);
    quickSort(A, pi + 1, high);
  }
}

public static int partition(int[] A, int low, int high) {
  int pivot = A[high];
  int i = (low - 1);
  for (int j = low; j < high; j++) {
    if (A[j] <= pivot) {
      i++;
      int tmp = A[i]; A[i] = A[j]; A[j] = tmp;
    }
  }
  int tmp = A[i+1]; A[i+1] = A[high]; A[high] = tmp;
  return i+1;
}
```

Erklärung:

→ Siehe nächste Seite für Ablauf-Erklärung mit Beispiel.

QuickSort (rekursiv) – Seite 2 (Erklärung)

Erklärung:

Ablauf: Wähle ein Pivot-Element, partitioniere das Array so, dass kleinere links und größere rechts stehen, dann rufe QuickSort rekursiv auf beide Teile.

Beispiel: [10,7,8,9,1,5], Pivot=5

1. Partition → [1,5,10,7,8,9]

2. Links=[1], Rechts=[10,7,8,9]

3. Rekursiv sortieren → [1,5,7,8,9,10]

Komplexität: $O(n \log n)$ durchschnittlich, $O(n^2)$ im Worst Case (schlechtes Pivot).

Typische Fehler: Partition falsch implementiert, Endlosschleifen bei falschen Indizes.

Prüfungstipp: Immer „Pivot – Partition – Rekursion“ sagen.